

0.76.9. La fonction `tourne_carre()` — un carré sans arrêter le jeu

`#include <tourne_carre>` : `tourne_carre(numero, x, y, tuile, cote, vitesse)`, appelée à chaque image, fait tourner une tuile sans fin, sans bloquer.

1. Ce qu'il faut comprendre

Ce qu'elle fait : contrairement à `carre()`, `tourne_carre(...)` **ne bloque pas** : on l'appelle à **chaque image**, et elle fait un pas quand c'est l'heure. Le jeu continue autour. La tuile tourne sans fin : `cote` pas à droite, en bas, à gauche, en haut.

La ligne à écrire : `#include <tourne_carre>`.

Ses arguments : un numéro (0 à 3 : la console retient où en est chaque tuile), le départ, la tuile, le côté (négatif : part vers la gauche), la vitesse en ms.

Ce qu'elle coûte : environ **470 octets**, plus la police entière.

Essaie : une deuxième tuile, avec le numéro 1.

On retrouve `tourne_carre` dans 5 leçons

[0.76.10](#), [0.76.11](#), [0.76.12](#), [0.76.13](#), [0.76.14](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1314 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
tourne_carre() 410 o ← main() – une fonction fournie par le langage
main()         33 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()        x 2    26 o sur place, dans tourne_carre() + AdresseCase (10 o)
effacer()      x 1    15 o sur place, dans tourne_carre() + AdresseCase (10 o), EffacerCases (12 o)
```

```
image()      x 1      3 o sur place, dans main() + AttendreImage (28 o)
images()     x 1      2 o sur place, dans tourne_carre()
Tes fonctions appelées :
tourne_carre() x 1      26 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <tourne_carre> : un carré sans arrêter le jeu ----

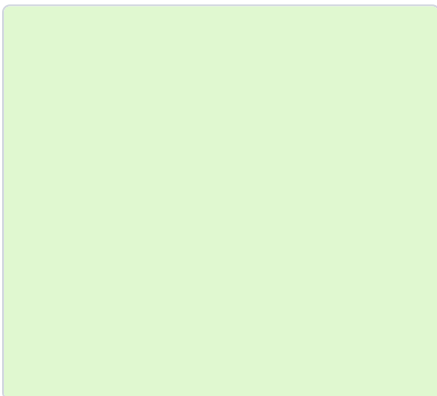
#include <tourne_carre>    // tourne_carre() : un pas quand c'est l'heure, sans bloquer
#include <ALPHABET>
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        tourne_carre(0, 8, 6, ALPHABET[0], 4, 100);    // départ (8, 6), côté 4, 100 ms par pas
    }
}
```

3. Ce qui se passe, étape par étape

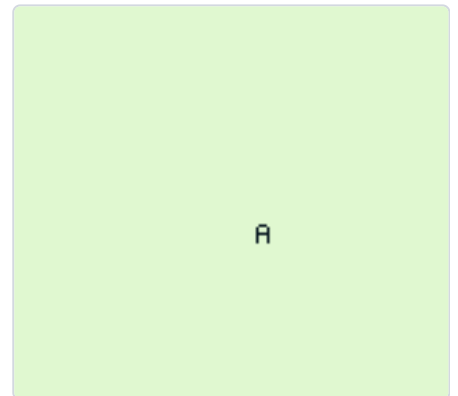
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



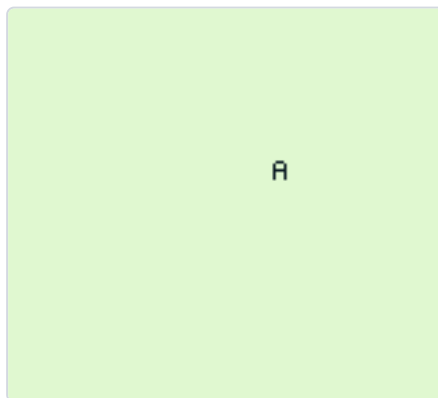
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui tourne en carré sans jamais s'arrêter.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A touche les quatre coins

✓ et il continue : plus d'un tour

Ensuite : **0.76.10. Un déclencheur en (0, 0) : le B tourne en carré — en simple** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++